A Single-Port SRAM Compiler for ASAP7 PDK

*Shao-Chien Lu, Yi-Chia Wu, Yu-Cheng Lin, Rung-Bin Lin**

Department of Computer Science and Engineering, Yuan Ze University, Taiwan
csrlin@saturn.yzu.edu.tw

ABSTRACT

This paper presents a fully automated single-port SRAM compiler for the ASAP7 PDK. The proposed compiler generates parameterized SRAM macros with a butterfly architecture and a hybrid implementation flow that combines custom memory arrays with synthesized control logic. It supports SRAM sizes from 32-bit to 131-kbit, enabling flexible exploration across a wide range of capacities. Characterization of a 64-kbit SRAM macro under the TT corner at 25°C shows an access time of 369 ps and a density of 27.58 bit/um^2. The area and access timing performance are comparable to that of handcrafted SRAMs provided by the PDK.

1. INTRODUCTION

With the rapid evolution of artificial intelligence (AI), the "Memory Wall" [1,2] has emerged as a critical bottleneck in high-performance computing (HPC). As AI workloads demand increasingly higher memory bandwidth, the gap between processor speed and memory access latency continues to grow. To address this challenge, the need for high-density, low-latency on-chip memory is growing exponentially.

Static Random Access Memory (SRAM) is the dominant on-chip memory technology due to its superior access time and seamless integration with standard CMOS processes. However, SRAM design in advanced FinFET nodes introduces unique challenges, including width quantization [3] and stringent design rules [4], which complicate the optimization of area, power, performance, reliability, etc.

Among various SRAM architectures, Single-Port (SP) SRAM based on a 6-transistor (6T) bitcell offers the higher bit density, as it requires a smaller cell area and simpler control logic compared to 2-Port (2P) or Dual-Port (DP) SRAM, which typically employ 8 or more transistors per bitcell. This density advantage makes SP-SRAM particularly attractive for large on-chip caches in HPC applications [5].

Significant research efforts have been devoted to improving SP-SRAM performance in FinFET technologies. Yabuuchi *et al.* [6], Chen *et al.* [7], and Chang *et al.* [8,9] demonstrated high-k/metal-gate SRAM macros with wordline overdrive assist techniques in FinFET processes, achieving improved minimum operating voltage V_{min} and read access time. Clinton *et al.* [5] proposed a 7 nm L1 cache SP-SRAM compiler for HPC applications, employing a folded macro architecture to reduce bitline length. Their design incurs a 15% area overhead but achieves a 15%~20% reduction in access time.

The 6T bitcell can also operate with time-multiplexed wordline pulses to realize pseudo 2-Port (P2P) or pseudo Dual-Port (PDP)-SRAM functionality. Yabuuchi *et al.* [10] implemented a 16 nm P2P-SRAM using 6T bitcells with double pumping, achieving 313 ps read access time. Haraguchi *et al.* [11] developed a folded P2P-SRAM macro operating at 4.3 GHz. Nautiyal *et al.* [12] utilized 6T bitcells to build an area- and power-saving PDP-SRAM in 16 nm FinFET technology for graphics processor. More recently, Tanaka *et al.* [13] proposed a 2-read-write PDP-SRAM in 3 nm FinFET technology with metal wire resistance tracking and sequential access-aware dynamic power reduction, demonstrating high bit density and improved energy efficiency.

Despite these advances, a critical gap remains in the design ecosystem. The ASAP7 PDK [14] has been widely adopted by the research community for FinFET technology exploration, yet it lacks a publicly available, robust SRAM compiler capable of generating parameterized SRAM macros. This limitation prevents the ASAP7 PDK and its standard cell libraries from being adopted for embedded and system-on-chip applications. .

In this paper, we address this gap by presenting the SP-SRAM compiler for the ASAP7 PDK. The main contributions of this work are summarized as follows:

- To the best of our knowledge, this is the first SP-SRAM compiler for the ASAP7 PDK, enabling the automated generation of parameterized SRAM macros.
- Supporting for configurable SRAM sizes from 32-bit to 131-kbit, facilitating flexible design exploration.
- Achieving an access time of 369 ps and a density of 27.58 bit/um^2 for a 64-kbit SRAM macro.

The remainder of this paper is organized as follows. Section 2 describes the SRAM macro architecture and details the bank-and-row organization. Section 3 presents the implementation results and performance evaluation. Finally, Section 4 concludes the paper.

2. ASAP7 PDK SP SRAM COMPILER

2.1. SP SRAM Architecture

Our compiler adopts a bank-and-row architecture. The macro is partitioned into multiple banks, and each bank is further divided into rows. As shown in Fig. 1, the wordlines (WLs) run vertically and the bitlines (BLs) run horizontally. This organization shortens the local interconnections, balances the delay, and keeps the layout compact.

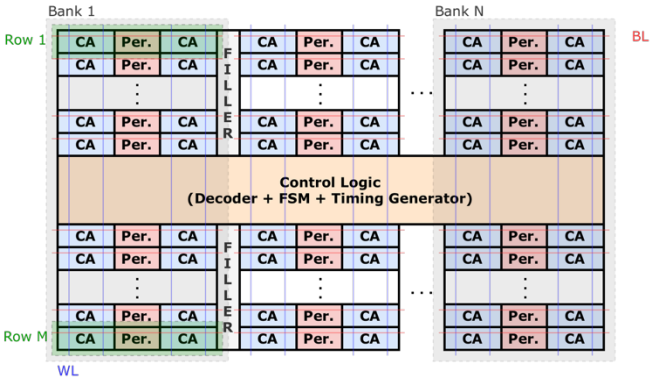


Fig.1. The proposed bank-and-row SRAM architecture.
* CA: cell array, Per.: peripheral circuitry

The SRAM I/O interface is summarized in Table I. The design uses a synchronous clock (CLK) for positive-edge-triggered operation, a chip enable (CEN) for macro selection, and a reset (RSTN) for initialization. Separate write enable (WEN) and output enable (OEN) control the data path direction.

2.2. Operational Timing

Figure 2 shows the timing of reset, read, and write operations. When RSTN is asserted, the internal state returns to IDLE. During a write cycle, WEN is low while OEN is high, and data is latched at the rising edge of CLK. During a read cycle, OEN is low and the sensed data appears on Q in the same cycle. The interface supports back-to-back access without dead cycles.

Table I. SRAM IO Pin Definitions.

Pin Name	Direction	Description
CLK	I	System clock (positive edge-triggered)
CEN	I	Chip enable (active-low)
WEN	I	Write enable (active-low)
OEN	I	Output enable (active-low)
RSTN	I	Asynchronous reset (active-low)
A	I	Address bus
D	I	Data input bus
Q	O	Data output bus

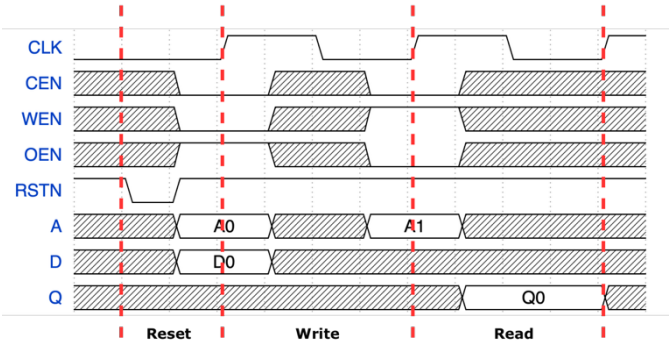


Fig. 2. Timing waveforms for reset, read, and write cycles.

2.3. Row Organization

Figure 3 illustrates the row organization within a bank. Each row contains eight bitcell segments arranged into left and right sub-arrays. One address bit selects the sub-array, and two additional bits select one of the four segments in that sub-array.

To improve area efficiency, the precharge circuit, sense amplifier, write driver, SR latch, and tri-state inverter are shared within each row. The wordlines are shared inside each sub-array, while the bitlines are kept local to each segment to limit parasitic capacitance and reduce access delay and dynamic power.

The internal control signals include sense amplifier enable (sae), write enable (wrena/wrenan), bitline precharge (blprechln/blprechrn), and output enable (oena/oenan). The input signals (D) and output signals (Q) are routed in parallel across all rows through Metal 4. During readout, a tri-state inverter ensures that only the selected row drives the shared output bus, thereby avoiding bus contention.

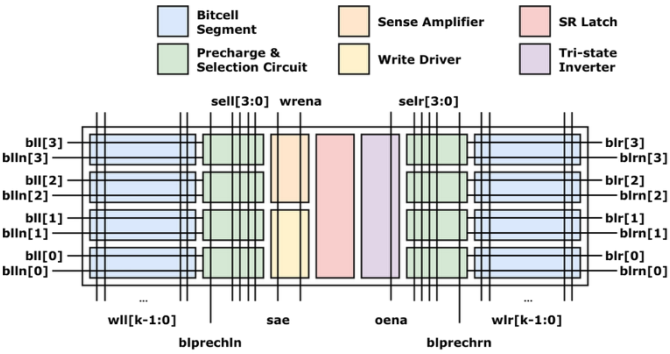


Fig. 3. Row organization.

2.4. Control Logic and Decoder

The control logic generates the timed signals required to coordinate the rows. We implement the control logic as a parameterized Verilog module, so the compiler can adapt to different numbers of banks and rows.

Unlike the full-custom row circuitry, the control logic is synthesized from the ASAP7 standard-cell library and laid out with an automatic place-and-route flow. We use Synopsys Design Compiler for logic synthesis and Cadence Innovus for physical implementation. This hybrid flow improves portability across SRAM configurations and reduces development time. Furthermore, future adaptation to other memory organizations, such as configuring a P2P-SRAM, by simply adjusting the behavioral Verilog code, thereby eliminating the need to redesign the control circuit from scratch.

The control logic also includes a parameterized address decoder that generates the sub-array and segment select signals. Its I/O interface is summarized in Table II.

Table II. Control block IO interface and functional descriptions.

Pin Name	Direction	Description
CLK	I	System clock (positive edge-triggered)
CEN	I	Chip enable (active-low)
WEN	I	Write enable (active-low)
OEN	I	Output enable (active-low)
RSTN	I	Asynchronous reset (active-low)
A	I	Address bus
sae	O	Sense amplifier enable
wrena	O	Write driver enable
wrenan	O	Write driver enable bar
oena	O	Output tri-state inverter enable
oenan	O	Output tri-state inverter enable bar
blprechln	O	Left sub-array precharge enable (active-low)
blprechrn	O	Right sub-array precharge enable (active-low)
wll	O	Left wordline bus
wlr	O	Right wordline bus
sell	O	Left sub-array column selection
selr	O	Right sub-array column selection

The address bus (A) is organized into several fields. The most significant bits select the banks, the next bit selects the left or right sub-array, the following two bits select one of the four bitcell segments, and the remaining bits drive the row decoder.

The control path uses delay chains to sequence write and read operations: during write, precharge is disabled and the write drivers are enabled before the target wordline is asserted; during read, the bitlines are precharged first and the sense amplifier is enabled after wordline activation. We do not use a replica bitline, since it adds hardware overhead that is hard to justify for a compact SRAM compiler [15].

The primary drawback of adopting a semi-custom flow for implementing control logic is the area overhead caused by the diffusion breaks within the standard cells and the higher cell count.

3. EXPERIMENTAL RESULTS

3.1. Layout

Figure 4 shows a partial layout view of the 256-word×64-bit×4-bank SRAM macro. Because of the limited figure resolution, only a cropped view is shown here to highlight the detailed placement. The layout demonstrates that the cell array, peripheral circuitry, and control logic are tightly integrated in a compact floorplan.

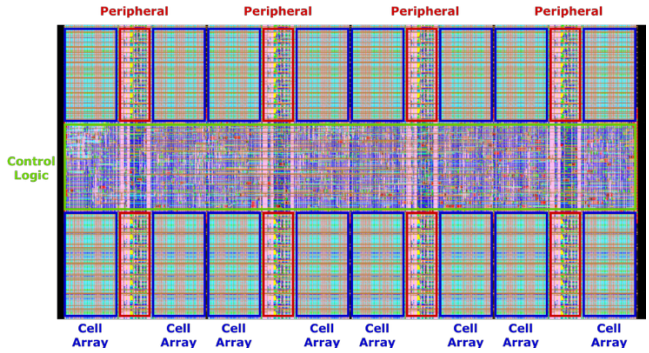


Fig. 4. Partial layout view of the SRAM macro.

3.2. Access Time and Area Comparison

The generated SRAM macros were characterized by SiliconSmart [16], a characterization tool from Synopsys, under TT corner and 25°C. We considered three representative capacities, 16-kbit, 32-kbit, and 64-kbit, and compared them with the corresponding SRAM macros accompanied by the ASAP7 PDK.

Table III summarizes the timing results obtained from characterization. Our compiler reduces both rise and fall access times in most configurations. This indicates that the automated timing search effectively tunes the internal control chain. The 1024-word, 64-bit instance shows a larger access time because of the increased number of buffers in the delay chain.

Table III. Timing performance comparison.

Instance	Type	PDK (ns)	Ours (ns)	Diff. (%)
256-word ×64-bit (16-kbit)	Rise delay	0.250	0.216	**-13.6**
	Rise trans.	0.072	0.064	**-11.1**
	Fall delay	0.252	0.216	**-14.3**
	Fall trans.	0.057	0.048	**-15.8**
512-word ×64-bit (32-kbit)	Rise delay	0.270	0.245	**-9.3**
	Rise trans.	0.070	0.063	**-10.0**
	Fall delay	0.264	0.246	**-6.8**
	Fall trans.	0.053	0.048	**-9.4**
1024-word ×64-bit (64-kbit)	Rise delay	0.297	0.369	24.2
	Rise trans.	0.068	0.061	**-10.3**
	Fall delay	0.287	0.367	27.9
	Fall trans.	0.058	0.047	**-19.0**

* Note: Input slew = 0.04 ns, output load cap. = 0.04608 pF.

Table IV summarizes the physical implementation results. Our modular floorplan keeps the macro height fixed and 1.2% smaller than the PDK counterpart. The total area is slightly larger, with an overhead at most 4.3%. This is a reasonable trade-off given that the PDK SRAMs are highly customized.

Table IV. Physical dimension and area comparison.

Instance	Dimension	PDK	Ours	Diff. (%)
256-word ×64-bit (16-kbit)	Width (um)	9.61	10.15	5.6
	Height (um)	77.80	76.84	**-1.2**
	Area (um^2)	747.66	779.93	4.3
512-word ×64-bit (32-kbit)	Width (um)	16.41	17.06	4.0
	Height (um)	77.80	76.84	**-1.2**
	Area (um^2)	1276.70	1276.77	0.0
1024-word ×64-bit (64-kbit)	Width (um)	30.35	30.89	1.8
	Height (um)	77.80	76.84	**-1.2**
	Area (um^2)	2361.23	2373.59	0.5

To further illustrate the flexibility of the proposed compiler, we evaluated several aspect ratios by varying the number of banks (b) and the number of wordlines in each sub-array (k) while keeping the total capacity fixed at 16-kbit, 32-kbit, or 64-kbit. The resulting rise and fall delays are listed in Table V.

Overall, $b = 2$ provides the best balance between layout regularity and timing. When $b = 1$, the macro becomes tall and narrow, which lengthens the vertical bitlines and increases bitline parasitic capacitance. When $b \geq 2$, the macro becomes wider, and the capacitance on the shared global data bus (D/Q) increases with the number of banks. The larger capacitive load on the Metal 4 routing slows signal propagation, so the compiler must calibrate the timing chain to preserve reliable sensing and data driving.

Table V. Timing of the SRAMs with different aspect ratios.

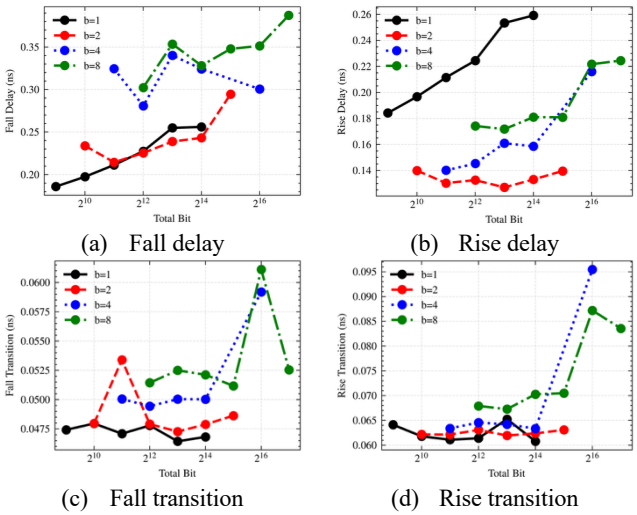
Instance	Bank (b)	Rise Delay (ns)	Fall Delay (ns)
256-word ×64-bit (16-kbit)	1	0.192	**0.193**
	2	**0.159**	0.196
	4	0.163	0.290
	8	0.187	0.284
512-word ×64-bit (32-kbit)	1	0.224	**0.224**
	2	**0.145**	0.257
	4	0.174	0.264
	8	0.190	0.289
1024-word ×64-bit (64-kbit)	1	0.232	0.234
	2	**0.150**	**0.212**
	4	0.180	0.293
	8	0.177	0.305

* Note: Input slew = 0.04 ns, output load cap. = 0.04608 pF.

To assess scalability, we characterized SRAM macros from 512-bit to 131-kbit under four bank configurations, $b = 1$, 2, 4, and 8. The trends of rise delay, fall delay, rise transition, and fall transition are summarized in Fig. 5.

As shown in Fig. 5(b), the rise delay is minimized at $b = 2$, with an average value of about 0.134 ns across the evaluated capacities. This configuration consistently outperforms the single-bank design, whose rise delay averages 0.222 ns. In contrast, Fig. 5(a) shows that the fall delay increases as the bank count grows. The $b = 1$ configuration achieves the lowest fall delay, while $b = 4$ and $b = 8$ incur a delay penalty, reaching average values of 0.314 ns and 0.345 ns, respectively.

The transition times are relatively stable for $b = 1$ and $b = 2$, as shown in Fig. 5(c) and Fig. 5(d). For larger bank counts, however, both transition times increase slightly, which is mainly caused by the heavier load on the shared global data bus. Overall, $b = 2$ provides the best balance between access speed and scalability.



(a) Fall delay (b) Rise delay

(c) Fall transition (d) Rise transition

Fig. 5. Timing characteristics at output Q versus bank count and memory capacity.

4. CONCLUSION

We presented a fully automated single-port SRAM compiler for the ASAP7 PDK. The proposed flow combines a bank-and-row memory organization with synthesized control logic to generate parameterized SRAM macros ranging from 32 bits to 131 kbits. The so-generated

SRAMs have comparable area and access timing performance to the SRAMs macros enclosed in the PDK. For the 64-kbit macro, the measured access time is 369 ps and the density reaches 27.58-bit/um^2. Overall, the results show that the proposed compiler offers a scalable SRAM design flow for ASAP7-based technology exploration, and future work will extend the flow toward additional memory organizations and broader architectural support.

REFERENCES

[1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH computer architecture news*, vol. 23, no. 1, pp. 20–24, 1995.

[2] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "Ai and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.

[3] D. Burnett, S. Parihar, H. Ramamurthy, and S. Balasubramanian, "FinFET SRAM design challenges," *IEEE International Conference on IC Design & Technology*, 2014, pp. 1–4.

[4] H. Kawasaki et al., "Challenges and solutions of FinFET integration in an SRAM cell and a logic circuit for 22 nm node and beyond," *IEEE International Electron Devices Meeting*, 2009, pp. 1–4.

[5] M. Clinton, R. Singh, M. Tsai, S. Zhang, B. Sheffield, and J. Chang, "A 5GHz 7nm L1 cache memory compiler for high-speed computing and mobile applications," *IEEE International Solid-State Circuits Conference*, 2018, pp. 200–201.

[6] M. Yabuuchi et al., "16 nm FinFET High-k/Metal-gate 256-kbit 6T SRAM macros with wordline overdriven assist," *IEEE International Electron Devices Meeting*, 2014, pp. 3–3.

[7] Y.-H. Chen et al., "A 16 nm 128 Mb SRAM in High-k Metal-Gate FinFET Technology With Write-Assist Circuitry for Low-VMIN Applications," *IEEE Journal of Solid-State Circuits*, vol. 50, no. 1, pp. 170–177, 2014.

[8] J. Chang et al., "12.1 a 7nm 256mb sram in high-k metal-gate finfet technology with write-assist circuitry for low-v min applications," *IEEE International Solid-State Circuits Conference*, 2017, pp. 206–207.

[9] J. Chang et al., "A 3nm 256mb sram in finfet technology with new array banking architecture and write-assist circuitry scheme for high-density and low-v min applications," *IEEE Symposium on VLSI Technology and Circuits*, 2023, pp. 1–2.

[10] M. Yabuuchi et al., "A 6.05-Mb/mm 2 16-nm FinFET double pumping 1W1R 2-port SRAM with 313 ps read access time," *IEEE Symposium on VLSI Circuits*, 2016, pp. 1–2.

[11] M. Haraguchi et al., "A 3 nm-FinFET 4.3 GHz 21.1 Mb/mm 2 Double-Pumping 1-Read and 1-Write Psuedo-2-Port SRAM With a Folded Bitline Multi-Bank Architecture," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 1, pp. 197–204, 2024.

[12] V. Nautiyal, G. Singla, L. Gupta, S. Dwivedi, and M. Kinkade, "An ultra high density pseudo dual-port SRAM in 16nm FINFET process for graphics processors," *IEEE International System-on-Chip Conference*, 2017, pp. 12–17.

[13] T. Tanaka et al., "A 3nm Fin-FET 19.87-Mbit/mm 2 2RW Pseudo Dual-Port 6T SRAM with High-R Wire Tracking and Sequential Access Aware Dynamic Power Reduction," *IEEE Symposium on VLSI Technology and Circuits*, 2024, pp. 1–2.

[14] L. T. Clark et al., "ASAP7: A 7-nm finFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.

[15] C. DC Arandilla and J. A. R. Madamba, "Comparison of replica bitline technique and chain delay technique as read timing control for low-power asynchronous SRAM," *Fifth Asia Modelling Symposium*, 2011, pp. 275–278.

[16] SiliconSmart. https://www.synopsys.com/implementation-and-signoff/signoff/siliconsmart.html